

Laboratory Work #1

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee

1. Superkeys: {EmpID}, {SSN}, {Email}, {EmpID;SSN}, {EmpID; Name}, {SSN, Email}
2. {EmpID}, {SSN}, {Email}
3. I would choose EmpID, because it can't change like email and it doesn't contain personal information
4. Yes. In practice, several employees often share one work number

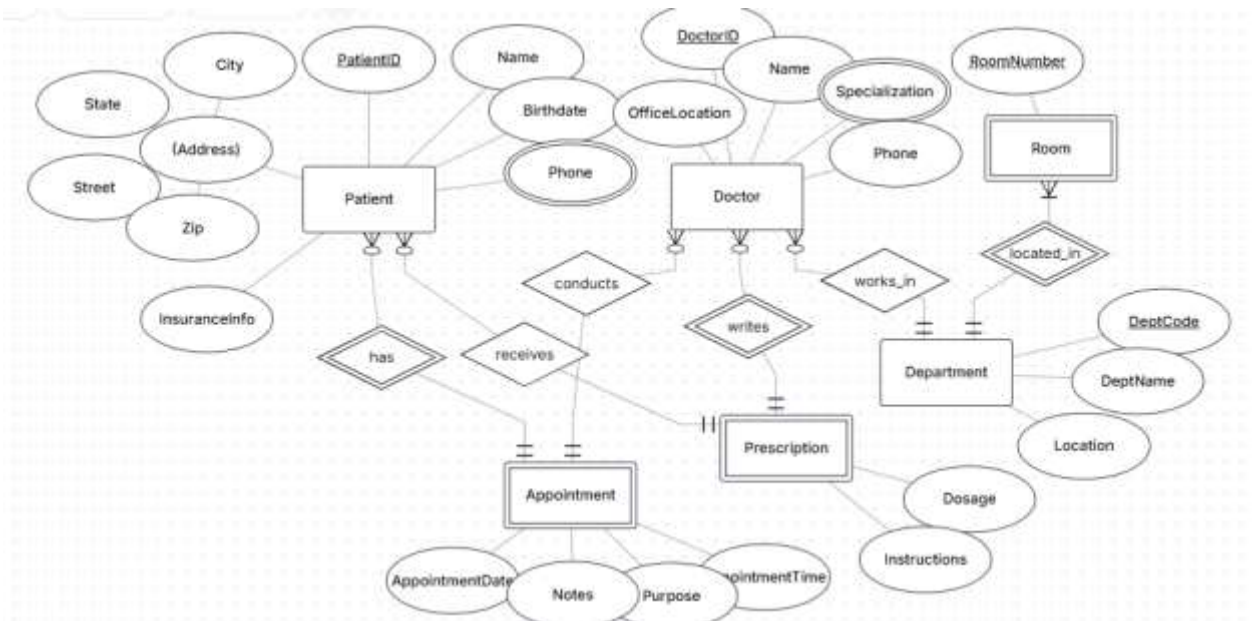
Relation B: Course Registration

1. The minimum attributes: {StudentID, CourseCode, Section, Semester, Year}
2. Why each attribute is needed:
StudentID — otherwise the record isn't tied to a specific student, and different students wouldn't be able to take the same course.
CourseCode — a student can take different courses at the same time (in the same semester).
Section — the rule explicitly says: the restriction is only on the same section in the same semester, so a student can be in different sections of the same course.
Semester + Year — both are required because Semester values (like "Fall") repeat every year; without Year, "Fall" 2024 and "Fall" 2025 would be indistinguishable.
3. None. Grade and Credits are dependent attributes, not candidates for uniqueness.

Task 1.2: Foreign Key Design

Foreign key	References
Student.StudentID	Professor.ProfID
Professor.Department	Department.DeptCode
Course.DepartmentCode	Department.DeptCode
Department.ChairID	Professor.ProfID
Enrollment.StudentID	Student.StudentID
Enrollment.CourseID	Course.CourseID

Task 2.1: Hospital Management System



Entities

- **Patient** — Strong
- **Doctor** — Strong
- **Department** — Strong
- **Appointment** — Strong (associative, between Patient and Doctor)
- **Prescription** — Strong (associative, between Doctor and Patient)
- **Room** — Weak — room numbers repeat across departments (101 in Cardiology ≠ 101 in Neurology); composite key = (RoomNumber, DeptCode)

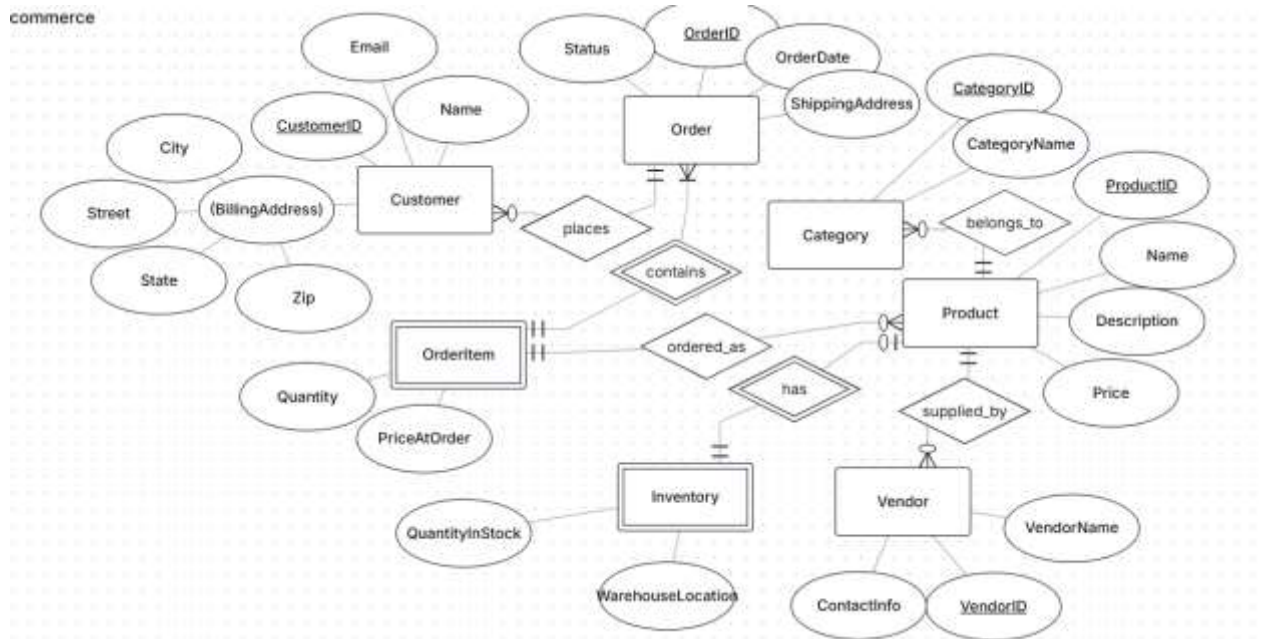
Attributes

- Patient: PatientID (simple, key), Name (composite: First/Last), BirthDate (simple), Address (composite: Street/City/State/Zip), PhoneNumbers (multi-valued), InsuranceInfo (composite), Age (derived, from BirthDate)
- Doctor: DoctorID (key), Name (composite), Specializations (multi-valued), PhoneNumber (simple), OfficeLocation (simple)
- Department: DeptCode (key), DeptName, Location
- Appointment: AppointmentID (key), DateTime, Purpose, Notes
- Prescription: PrescriptionID (key), Medication, Dosage, Instructions
- Room: RoomNumber (partial key) + DeptCode (owner key) = composite key of the weak entity

Relationships & cardinalities

- Patient — Appointment: 1:N, total participation on the Appointment side
- Doctor — Appointment: 1:N, total participation on the Appointment side
- Doctor — Prescription — Patient: M:N (via Prescription as an associative entity)
- Department — Doctor: 1:N
- Department — Room: 1:N, identifying relationship (weak entity), total participation on the Room side

Task 2.2 — E-commerce Platform



Entities

- **Customer** — Strong
- **Order** — Strong
- **Product** — Strong
- **Category** — Strong
- **Vendor** — Strong
- **OrderItem** — Weak — meaningless without its Order; key = (OrderID, ProductID)
- **Review** — Strong (associative, Customer ↔ Product)
- **Inventory** — Strong (or an attribute of Product, depending on scope)
- **Address (Shipping/Billing)** — Weak, owned by Customer

1) Weak entity and justification

OrderItem is the weak entity. It has no natural stand-alone identifier — a row only makes sense in the context of one specific order. Its identifying (composite) key is {OrderID, ProductID}, where OrderID comes from the owner entity Order.

2) M:N relationship requiring attributes

Order — Product, realized through the associative entity OrderItem, carrying its own attributes: Quantity and PriceAtOrderTime. This is a textbook case where an M:N relationship must carry attributes, since the price at the time of purchase can differ from the product's current price — it cannot be reduced to a plain relationship without an intermediate entity.

Other relationships

- Product — Category: N:1
- Product — Vendor: N:1 (or M:N if a product can be supplied by several vendors)
- Customer — Review — Product: M:N via Review (Rating, Comment, Date)

- Product — Inventory: 1:1
- Customer — Address: 1:N (billing/shipping distinguished by an AddressType attribute)

Task 4.1 — StudentProject

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1) Functional dependencies

StudentID → StudentName, StudentMajor

ProjectID → ProjectTitle, ProjectType, SupervisorID

SupervisorID → SupervisorName, SupervisorDept

(StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

2) Problems

- Redundancy: StudentName/StudentMajor repeat for every project a student works on; SupervisorName/SupervisorDept repeat for every student under the same supervisor.
- Update anomaly: if a supervisor changes department, every row mentioning them must be updated — missing even one row leaves inconsistent data.
- Insert anomaly: a new supervisor cannot be added until they are assigned to at least one student/project — there is no row to hold their data otherwise.
- Delete anomaly: if a student is the sole member of a project and their row is deleted, the project's own data (ProjectTitle, ProjectType) and the supervisor's data are lost as well.

3) 1NF

No violations — every attribute is atomic (no repeating groups or multi-valued fields). The table is already in 1NF.

4) 2NF

Primary key: {StudentID, ProjectID}

Partial dependencies (depend on only part of the composite key):

StudentID → StudentName, StudentMajor

ProjectID → ProjectTitle, ProjectType, SupervisorID

2NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)

StudentProjectRole(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5) 3NF

Transitive dependency inside Project: ProjectID → SupervisorID → SupervisorName, SupervisorDept

Final 3NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Supervisor(SupervisorID, SupervisorName, SupervisorDept)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)

StudentProjectRole (*StudentID*, *ProjectID*, Role, HoursWorked, StartDate, EndDate)

(*Bold + underlined = primary key; italic = foreign key*)

Task 4.2 — CourseSchedule

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1) Primary key (the tricky part)

Since a student can be enrolled in multiple course sections, and CourseID here represents one specific section (with one instructor, one time slot, one room), the minimal key that uniquely identifies a row is:

PK(StudentID, CourseID)

2) Functional dependencies

StudentID → StudentMajor

CourseID → CourseName, InstructorID, TimeSlot, Room

InstructorID → InstructorName

Room → Building (rooms are unique campus-wide)

(Room, TimeSlot) → CourseID (one section per room per time slot)

(InstructorID, TimeSlot) → CourseID (an instructor teaches one section at a time)

3) BCNF check

Candidate key: {StudentID, CourseID}. BCNF rule: every determinant must be a superkey.

- CourseID → CourseName, InstructorID, TimeSlot, Room — CourseID is NOT a superkey (does not determine StudentID) → BCNF violation.
- Room → Building — Room is not a superkey → BCNF violation.
- InstructorID → InstructorName — not a superkey → BCNF violation.

Conclusion: the table is NOT in BCNF.

4) BCNF decomposition

Student(StudentID, StudentMajor)

Instructor(InstructorID, InstructorName)

RoomInfo(Room, Building)

Course(CourseID, CourseName, InstructorID, TimeSlot, Room)

Enrollment(StudentID, CourseID)

5) Potential information loss

- Lossless join: preserved. The decomposition follows the real FDs and every pair of tables shares a key attribute (InstructorID, Room, CourseID), so the original table can be reconstructed exactly via JOIN.
- Dependency preservation: partially lost. (Room, TimeSlot) → CourseID and (InstructorID, TimeSlot) → CourseID are no longer enforced automatically by any single table after decomposition — they must be enforced with an extra UNIQUE constraint on (Room, TimeSlot) and (InstructorID, TimeSlot) in the Course table, or with an application-level check.

Part 5: Design Challenge — Student Clubs

1) Entities and relationships

- **Student** — StudentID, Name, Email
- **Club** — ClubID, Name, Description
- **Faculty (Advisor)** — FacultyID, Name, Department
- **Membership (associative, M:N Student↔Club)** — StudentID, ClubID, JoinDate
- **Event** — EventID, ClubID, Name, DateTime
- **Attendance (associative, M:N Student↔Event)** — StudentID, EventID

Relationships & cardinalities

- Student — Club: M:N via Membership
- Faculty — Club: 1:N (one advisor can advise several clubs, each club has one advisor)
- Club — Event: 1:N
- Student — Event: M:N via Attendance
- Event — Room: N:1 via Reservation

2) Normalized relational schema

Student(StudentID, Name, Email)

Club(ClubID, Name, Description, AdvisorID)

Membership(StudentID, ClubID, JoinDate)

Advisor(AdvisorID, Name, Department)

Event(EventID, ClubID, Name, DateTime)

Attendance(StudentID, EventID)

3) Design decision with multiple valid options

Question: should a student's officer role (president, treasurer, etc.) be stored as a plain attribute inside Membership, or as a separate OfficerPosition entity?

A separate OfficerPosition entity with.

Reasoning: positions change over time — the same student might be treasurer one year and president the next. Membership just records the fact of joining a club, not a career inside it. A separate entity preserves a full history of positions without corrupting the membership record, and correctly answers questions like "who was president of the club in 2024?".

4) Example queries

- "Find all students who are officers in the Computer Science Club."
- "List all events scheduled for next week with their room reservations."
- "Find all clubs advised by a given faculty member, along with their current budget balance"

